

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 8

The Same Calls, Inside Your Own OS

Course: Operating Systems (Track B: xv6)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Purpose of the work

Lab 7 ran `fork`, `exec` and `wait` on Linux, on top of glibc and a kernel of some thirty million lines. The purpose of this laboratory work is to run the same three calls inside xv6 — the small operating system built during this course, compiled from source on this machine — and to find that they are the same calls: to write `user/run.c`, which performs the three steps of the shell with the child becoming `echo`, and then to make that child become `ls` and finally a program that does not exist; and to write `user/pingpong.c`, in which a parent and a child pass one byte back and forth through two pipes, so that the synchronisation Lab 7 observed from outside is built by hand from the inside. The optional `user/primes.c`, a chain of processes that sieves the primes below 35, was written as well.

All work was done in `~/xv6-riscv` at commit `06aad25`, cross-compiled with `riscv64-linux-gnu-gcc 15.2.0` and run under QEMU 10.2.1 emulating a three-hart RISC-V machine, on an Ubuntu 26.04.1 host. Each new program is one file `user/NAME.c` plus one line `$U/_NAME\` in the `UPROGS` list of the Makefile, beginning with a TAB and with nothing after the backslash; `make` then compiles it and `mkfs` copies it into the disk image `fs.img`. The experiments that required editing a program — the two `exec` variants of Part 1 — were made in place and then reverted, so the submitted `run.c` is the version the lab specifies; the two variants written to support a Part 2 answer were separate files, removed from `user/` and from `UPROGS` afterwards. In the figures the xv6 shell prompt is the bare `$` and the host prompt is shortened to `~/xv6-riscv$`. The complete session is in `lab8-transcript.log`.

Preparation

```
~/xv6-riscv$ sed -n '/UPROGS=/,/^$/p' Makefile | tail -4
        $U/_run\
        $U/_pingpong\
        $U/_primes\


~/xv6-riscv$ make clean ; make fs.img kernel/kernel
riscv64-linux-gnu-gcc ... -c -o user/run.o user/run.c
riscv64-linux-gnu-gcc ... -c -o user/pingpong.o user/pingpong.c
riscv64-linux-gnu-gcc ... -c -o user/primes.o user/primes.c
mkfs/mkfs fs.img README ... $U/_run $U/_pingpong $U/_primes
~/xv6-riscv$ make qemu
xv6 kernel is booting


hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 0.1 – The three UPROGS lines, a clean build, and the boot

The three programs compile without a warning of their own; the only two the build emits, about `LOAD segment with RWX permissions` in `_forktest` and `kernel`, are pre-existing messages of the linker that appear in an untouched tree as well. `fs.img` survives between runs, so every figure below was produced after `rm -f fs.img` and a fresh `make fs.img`, which is why the PIDs start from 3 each time.

Part 1. run, inside xv6

`user/run.c` is the Lab 7 program rewritten for a kernel with no glibc underneath it: there is no `execlp` to search a `PATH`, no `perror` and no `errno`, only the system calls of the kernel compiled a minute earlier. The three steps are unchanged.

Interesting Command Output:

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Figure 1.1 – Step 1.2: the child becomes echo

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
   ...
mystat         2 25 36376
find           2 26 42632
run            2 27 35888
pingpong       2 28 35968
primes         2 29 36456
console        3 30 0
parent: child 4 ended with status 0
```

Figure 1.2 – Step 1.3: two lines changed, and the child becomes ls

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

Figure 1.3 – Step 1.4: exec("nosuchprog", args)

Observations:

1. **Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Lab 7. What is the same?**

   The three steps and their order are identical; only the spelling differs, as Table 1.1 sets out.

Table 1.1 – The same three steps, in two operating systems

Step	xv6 (Lab 8)	Linux (Lab 7)
1. copy myself	`fork()`	`fork()`
2. become another program	`exec("echo", args)`	`execlp("ls", "ls", "-l", NULL)`
3. wait for the child	`wait(&status)`	`waitpid(pid, &status, 0)`

   What is the same is everything that matters, which is the behaviour of each step rather than its name. `fork` returns twice, 0 in the child and the child's PID in the parent — Figure 1.1 shows pid 3 and pid 4 printed by the two sides of the same `if`. `exec` replaces the program image and therefore returns only when it has failed, which is why `exec failed` appears in Figure 1.3 and in neither of the other two runs. `wait` blocks until the child is finished and hands back its exit status, and the parent's last line is printed after the child's output in all

three figures. Every property measured on Linux in Lab 7 reappeared unchanged on a kernel of a few thousand lines compiled on this machine, which is the whole point: these are not conveniences that glibc adds on top of an operating system, they are the interface of the operating system.

What differs is spelling and ambition. xv6's `exec` takes its argument vector as an array and does no `PATH` search — there is no `PATH` in xv6, so `"echo"` is a pathname and the call works only because `echo` sits in the root directory — whereas `execlp` is variadic and searches. xv6's `wait` takes only a status pointer and reaps any child, while `waitpid` can name one particular child and accepts flags. The status is plain here, left in `status` as it is, while Linux packs it into a word that `WEXITSTATUS` has to unpack. And there is no `perror` and no `errno` in xv6: the failure is announced by our own `printf`, because the kernel offers nothing else.

2. **Who prints "hello from the child": the code of `run`, or `echo`?**

   `echo` does. The sentence does not exist anywhere in `run.c` — what `run.c` contains is the vector `{ "echo", "hello", "from", "the", "child", 0 }`, five separate words and a terminating zero. The line in Figure 1.1 is assembled by `user/echo.c`:

```
int
main(int argc, char *argv[])
{
  int i;


  for (i = 1; i < argc; i++) {
    write(1, argv[i], strlen(argv[i]));
    if (i + 1 < argc) {
      write(1, " ", 1);
    } else {
      write(1, "\n", 1);
    }
  }
  exit(0);
}
```

Figure 1.4 – user/echo.c: the spaces and the newline are its work, not ours

So even the single spaces between the words and the newline at the end belong to `echo`, and it writes them to descriptor 1 — the same descriptor 1 that the whole of Part 3 of Lab 7 was about. Step 1.3 proves the same thing from the other direction: changing two lines to `exec("ls", args)` replaced the greeting with a directory listing, and that listing is plainly the work of `ls`. After a

successful `exec` there is no code of `run` left running in that process at all; what survives the call is the PID, the parent and the open descriptors, which is exactly why the parent in Figure 1.2 still reports on "child 4" although child 4 has been `ls` for some time.

3. **Why is the status 1 here and 127 on Linux? (Look at the `exit` line after `exec` in both programs.)**

   Because in both labs that status is produced by our own source, and the two sources were written with different numbers in them. Here the line below `exec` is `exit(1)`, so the parent printed 1; in Lab 7 the line below `execlp` was `exit(127)`, so the parent printed 127. The kernel chose neither value and could not have — it never saw the failing program, because there was no such program. The reason a number of ours gets through at all is the property from question 1: `exec` returns only on failure, so the `printf` and the `exit` beneath it are reached precisely when no new program exists to overwrite them.

   127 is not a better answer than 1, merely a conventional one: it is the "command not found" status that Unix shells report, so a Linux program meaning to behave like a shell adopts it deliberately. The xv6 shell implements no such convention, so the generic 1 is the natural choice here. The comparison is a useful reminder that an exit status is just a small number the program picks for itself, and that whatever meaning it carries is a convention between programs rather than a rule of the kernel.

Part 2. pingpong

The parent sends one byte to the child through `p2c`, the child prints its ping and sends the byte back through `c2p`, and the parent prints its pong. The four TODO lines of the skeleton are the two `read` and two `write` calls; each process closes the two ends it does not use before anything else happens.

Interesting Command Output:

```
$ pingpong
6: received ping
5: received pong
```

Figure 2.1 – The child (6) pings first, the parent (5) pongs second

Table 2.1 – Which end belongs to whom

Pipe	Parent keeps	Child keeps	Carries
`p2c`	`p2c[1]` (write)	`p2c[0]` (read)	the ping, parent → child
`c2p`	`c2p[0]` (read)	`c2p[1]` (write)	the pong, child → parent

Observations:

1. **Why do you need TWO pipes, and not one for both directions?**

   Because a pipe is one-directional. It has a read end and a write end, and bytes travel only from the second to the first, so a single pipe cannot carry the ping one way and the pong back the other: both processes would have to read and write the same channel, and nothing would prevent a process from reading the very byte it had just written. That is not a theoretical worry — it is what happens. A one-pipe version, identical except that `p[0]` and `p[1]` serve both directions, produced this:

```
$ pp1pipe
7: received pong
   (no new prompt: the program never ends)
```

Figure 2.2 – One pipe for both directions: the parent reads back its own byte

   Exactly one line was printed, and it is the **parent's**. The parent wrote its byte, immediately read from the same pipe, and received its own byte back — so it announced a pong that nobody had sent, and it did so without the child having run at all. The child, for its part, never received anything and is still blocked in `read`, which is why the parent's `wait(0)` never returned and xv6 never printed another prompt. With two pipes this is impossible by construction rather than by luck: each process closes the two ends it must not touch, so `p2c` can only carry bytes from parent to child and `c2p` only from child to parent, and a byte written into one of them can never be read back by its own writer. Table 2.1 is therefore not bookkeeping but the correctness argument.

2. **Nothing in your code says "wait until the child has replied". Which call makes the parent wait?**

   `read(c2p[0], &b, 1)`. A `read` on a pipe that holds no data blocks until either a byte arrives or every writing end has been closed, so the parent stops on that line and does not reach its `printf` until the child has actually written the byte back. That single blocking call is the entire synchronisation of the program, and the order of the two lines in Figure 2.1 is its consequence.

It is specifically **not** `wait(0)` that orders them, although it is tempting to think so, since it is the call with "wait" in its name. `wait` only reaps the finished child, and it runs after both messages have already been printed. Removing it entirely leaves the ordering untouched:

```
$ ppnowait
4: received ping
3: received pong
$ ppnowait
6: received ping
5: received pong
```

Figure 2.3 – The same program with wait(0) removed: ping still comes first

"ping" precedes "pong" in both runs, because what holds the parent back is the `read` and not the `wait`. The same mechanism works on the other side as well: the child opens by blocking in `read(p2c[0], &b, 1)` and so cannot print its ping before the parent has sent anything. The two blocking reads between them are what make the conversation strictly alternate, and they are the same mechanism that made `wc` wait for ever in Part 4 of Lab 7 — useful there, and load-bearing here.

Optional. primes

`user/primes.c` prints the primes from 2 to 35 with one process per prime: each stage reads its first number, which must be a prime, prints it, and forwards to the next stage only the numbers that this prime does not divide.

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

Figure 3.1 – All eleven primes below 35, each printed by a different process

Two points of the implementation are worth naming, because each of them was a trap:

- **The chain is a loop, not a recursion.** The first version called the stage function recursively in the forked child and was refused by the compiler with `error: infinite recursion detected [-Werror=infinite-recursion]`, since nothing in the source tells gcc that the recursive call happens only in a child that never returns. Letting the child take over the same loop with its own left-hand descriptor instead removes the problem altogether and costs no stack — which matters in xv6, where the user stack is a single 4096-byte page.
- **Every stage closes every end it does not use.** Each stage closes the read end of the pipe it writes to, the write end of the pipe it reads from, and its own left end when the numbers run out; the final `close(right[1])` is what gives the next stage end-of-file and lets the chain unwind from the far end backwards. Without it the pipeline would hang exactly like the deliberately broken `pipeline.c` of Lab 7, for exactly the same reason: a pipe counts writers, and a stage that still holds a writing end open is a writer that will never write.

Conclusion

The point of this laboratory work was that nothing new had to be learned, and that is the result it produced. The program of Lab 7 was rewritten for an operating system of a few thousand lines compiled on this machine, and the three calls behaved precisely as they had on Linux: `fork` returned twice, printing pid 3 and pid 4 from the two sides of one `if`; `exec` returned only when it had failed, which is why `exec failed` appeared in the `nosuchprog` run and in no other; and `wait` delivered a status that our own `exit` line had chosen — 1 here against 127 in Lab 7, a difference of convention between two sources and not a difference between two kernels. `echo` and not `run` printed "hello from the child", down to the spaces and the newline, because after a successful `exec` no code of `run` remains in that process; only its PID, its parent and its descriptors survive, which is why the parent could still report on "child 4" after child 4 had become `ls`.

`pingpong` then rebuilt from the inside the mechanism that Lab 7 had only observed. Two pipes are needed because a pipe is one-directional, and the one-pipe version demonstrated the failure instead of asserting it: the parent read back its own byte, announced a pong nobody had sent, and hung for ever waiting on a child still blocked in `read`. The call that synchronises the two processes turned out to be `read` and not

`wait`, which `ppnowait` settled by removing `wait(0)` and leaving the order of the output unchanged. `primes` extended the same idea to a chain of eleven processes and reproduced both lessons at once: close every end you do not use or the chain never ends, and build the chain with a loop rather than a recursion, because the compiler is right that a function calling itself unconditionally is an infinite recursion — it cannot know that `fork` has intervened.

Taken together with Lab 7, the two halves make one argument. The same seven calls — `fork`, `exec`, `wait`, `pipe`, `read`, `write` and `close` — are what a shell is made of, on a kernel of thirty million lines and on a kernel of three thousand. Lab 7 showed that they are all the shell has; Lab 8 showed that they are also all the operating system needs to offer.